



KOCAELİ ÜNİVERSİTESİ YAZILIM MÜHENDİSLİĞİ

**PROGRAMLAMA II DERSİ
PROJE RAPORU
PAC-MAN**

240229110 Enes DOĞRUL

31.05.2026

Dr. Öğr. Üyesi İrfan KÖSESOY

1. PROJENİN TANITIMI

Bu projede, klasik Pac-Man oyununun bir benzeri C++ programlama dili ve SFML 3.0.1 kütüphanesi kullanılarak geliştirilmiştir. Oyunda oyuncu, Pac-Man'i labirent içinde yönlendirerek noktaları toplar ve puan kazanır. Bu sırada dört hayalet, geliştirilen yapay zekâ algoritması sayesinde rastgele dolaşmak yerine oyuncuyu takip eder. Oyuncu bir hayaletle yakalandığında can kaybeder. Ancak labirentteki güç meyvelerinden biri yendiğinde hayaletler bir süreliğine savunmasız hâle gelir ve oyuncudan kaçır; bu sırada oyuncu onları yiyerek ek puan kazanabilir.

1.1. Projenin Amacı

Bu projenin amacı, derste öğrenilen programlama bilgilerinin gerçek ve oynanabilir bir uygulama üzerinde uygulanmasıdır. Bir oyun geliştirmek; ekrana görsel çizme, klavyeden gelen komutları okuma, karakterleri hareket ettirme, çarpışmaları kontrol etme ve yapay zekâ ile yol bulma gibi birçok farklı konunun bir arada çalışmasını gerektirir.

1.2. Projenin Kapsamı

Projenin kapsamı, çalışan bir Pac-Man oyununu temel özellikleriyle birlikte ortaya koymaktır. Bu kapsamda labirentte nokta toplama, hayaletlerin oyuncuyu kovalaması, güç meyvesiyle hayaletlerin kaçış moduna geçmesi, can ve puan takibi, yeni seviyeye geçiş ve yüksek skor gösterimi gibi özellikler geliştirilmiştir. Aşağıdaki tabloda, başlangıçta hedeflenen temel özellikler ve bunların gerçekleşme durumları yer almaktadır.

Hedeflenen Özellik	Durum
Labirentte nokta toplayarak puan kazanma	Tamamlandı
Oyuncuyu kovalayan hayaletler	Tamamlandı
Güç meyvesi ile hayaletlerin kaçması ve yenilebilmesi	Tamamlandı
Tüm noktalar yenildiğinde yeni seviye başlaması	Tamamlandı
En az iki farklı hayalet davranış kalıbı	Tamamlandı
Can, skor ve yüksek skor gösterimi	Tamamlandı
Ses efektleri ve başlangıç ekranı	Tamamlandı

Tablo 1. Hedeflenen özellikler ve gerçekleşme durumları

Tabloda da görüldüğü üzere, projenin başlangıcında hedeflenen tüm temel özellikler başarıyla gerçekleştirilmiştir. Bu yönüyle proje, başta belirlenen kapsamı eksiksiz bir biçimde karşılamaktadır.

2. KULLANILAN TEKNOLOJİLER

- Programlama Dili: C++ (C++17 standardı)
- Kullanılan Kütüphaneler: <iostream>, <fstream>, <cmath>, <optional> ve SFML 3.0.1
- Veri Yapıları: std::queue, std::vector
- Geliştirme Ortamı: macOS

3. KURULUM VE ÇALIŞTIRMA

3.1. Gereksinimler

- C++17 destekleyen bir derleyici (clang++ veya g++)
- SFML 3.0.1 kütüphanesinin kurulu olması

3.2. Klasör Yapısı

Oyunun çalışması için doku, ses ve font dosyalarının doğru konumlarda bulunması gerekir. Mevcut sürümde dosya yolları kaynak kod içinde sabit tanımlanmıştır:

```
PACMAN/  
textures/pacman/ -> Pac-Man ve hayalet görselleri (.png)  
resources/      -> Ses dosyaları (chomp, death, eat_ghost,  
                  power_pellet, siren)  
src/            -> arial.ttf ve highscore.txt
```

3.3. Derleme

```
clang++ -std=c++17 main.cpp -o pacman \  
-lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio
```

3.4. Kontroller

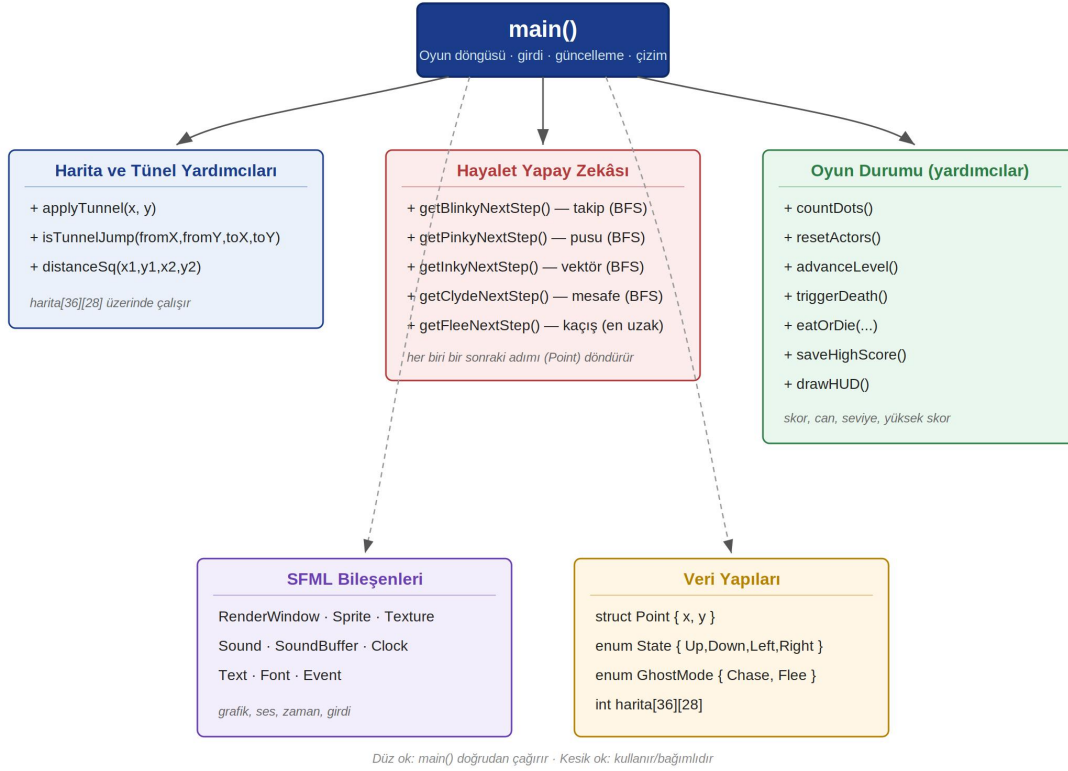
Oyun, klavye üzerinden yönlendirilir. Kullanılan tuşlar ve işlevleri aşağıdaki tabloda gösterilmektedir:

Tuş	İşlev
W	Yukarı hareket
A	Sola hareket
S	Aşağı hareket
D	Sağa hareket
ESC	Oyundan çıkış

Tablo 2. Oyun kontrolleri

4. SİSTEM MİMARİSİ VE TASARIM

Oyun, tek bir ana döngü etrafında çalışan modüler bir yapıya sahiptir. Aşağıdaki diyagramda programın temel bileşenleri; harita ve tünel yardımcıları, hayalet yapay zekâsı, oyun durumu yardımcıları, SFML bileşenleri ve veri yapıları olarak gruplanmıştır.



Şekil 1. Sistem mimarisi ve temel bileşenler

4.1. Harita

Labirent, 36 satır ve 28 sütundan oluşan iki boyutlu bir tam sayı dizisi (harita[36][28]) ile temsil edilir. Her hücredeki sayı, o karenin türünü belirtir:

Değer	Anlamı
0	Boş geçiş alanı
1	Duvar
3	Normal nokta (yem)
8	Güç meyvesi (büyük yem)
2 / 4 / 5 / 6 / 7	Pac-Man ve hayaletlerin başlangıç konumları

Tablo 3. Harita hücre değerleri ve anlamları

Oyun başında bu özel başlangıç değerleri (2, 4, 5, 6, 7) okunup ilgili karakterlerin konumuna atanır ve hücre boş alana (0) çevrilir. Haritanın ilk hâli ayrıca saklanır; böylece yeni seviyeye geçişte noktalar yeniden yüklenebilir.

4.2. Oyun Döngüsü

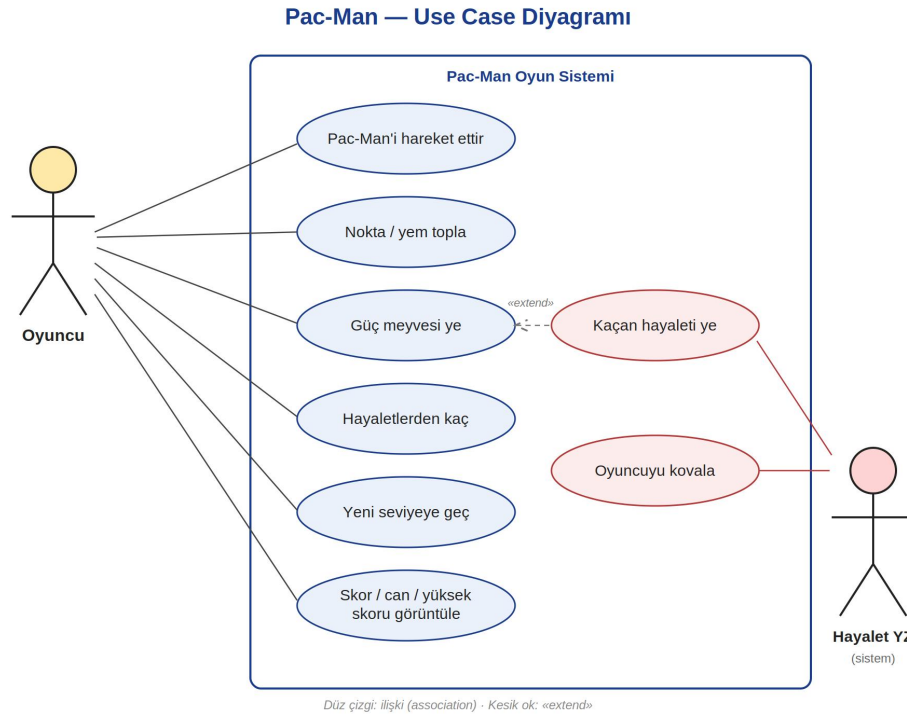
Program, pencere açık olduğu sürece çalışan bir ana döngü üzerine kuruludur. Her tur (frame) üç temel aşamadan oluşur: girdi okuma (klavye olayları), durum güncelleme (oyuncu ve hayaletlerin hareketi, çarpışma denetimi, skor) ve çizim (labirent, karakterler ve arayüzün ekrana basılması).

4.3. Delta-Time ile Hareket

Karakterlerin hareketi, her turda geçen gerçek süreye (delta-time, dt) bağlanmıştır. Bu sayede hareket hızı bilgisayarın FPS değerinden bağımsız hâle gelir; oyun hem hızlı hem yavaş sistemlerde aynı hızda akar. Karakterler kareden kareye doğrusal olarak (interpolasyonla) kaydırılır.

5. KULLANIM SENARYOLARI

Aşağıdaki kullanım senaryosu (use case) diyagramı, oyuncu ile sistem arasındaki temel etkileşimleri ve hayalet yapay zekâsının üstlendiği görevleri özetlemektedir.



Şekil 2. Pac-Man use case diyagramı

6. HAYALET YAPAY ZEKÂSI

Projenin en ayırt edici bölümü, dört hayaletin birbirinden farklı davranış kalıplarıdır. Hayaletlerin çoğu, hedef kareye giden en kısa yolu Genişlik Öncelikli Arama (BFS) algoritmasıyla bulur. BFS, labirent bir çizge (graph) olarak ele alındığında başlangıç karesinden

hedefe ulaşan en kısa adım dizisini garanti eder. Her hayaletin farklılığı, BFS'e verdikleri hedef karesinin nasıl belirlendiğinden kaynaklanır.

Hayalet	Davranış Kalıbı
Blinky	Doğrudan Pac-Man'in bulunduğu kareyi hedefler
Pinky	Pac-Man'in baktığı yöndeki 4 kare ilerisini hedefler; pusu kurarak önünü kesmeye çalışır.
Inky	Pac-Man'in 2 kare önünden Blinky'ye uzanan vektörü iki katına çıkararak hedef belirler;
Clyde	Pac-Man'e 8 kareden uzaktayken onu kovalar, yaklaşıncaya köşeye çekilir; çekingen davranır.

Tablo 4. Hayaletlerin davranış kalıpları

6.1. Kaçış (Flee) Modu

Oyuncu güç meyvesi yediğinde tüm hayaletler kaçış moduna geçer. Bu modda hayaletler BFS kullanmaz; bunun yerine komşu dört kare arasından Pac-Man'e olan uzaklığı en büyük olanı seçerek ondan kaçarlar. Kaçış süresi (yaklaşık yedi saniye) dolmadan iki saniye önce hayaletler yanıp söneren sürenin bitmek üzere olduğunu bildirir. Bu sırada Pac-Man bir hayalete dokunursa onu yer; yenen hayalet evine (başlangıç konumuna) geri gönderilir.

7. OYUN MEKANİKLERİ

7.1. Puanlama

- Normal nokta: 1 puan
- Güç meyvesi: 50 puan
- Kaçan hayalet yeme: art arda 200, 400, 800 ve 1600 puan (her güç meyvesi süresinde katlanarak artar)

7.2. Can ve Yeniden Doğuş

Oyuncu üç canla başlar. Bir hayalete yakalandığında ölüm animasyonu oynatılır, can sayısı bir azalır ve tüm karakterler başlangıç konumlarına döner. Canlar bittiğinde Oyun Bitti ekranı gösterilir.

7.3. Seviye İlerlemesi

Haritadaki tüm noktalar ve güç meyveleri yenildiğinde yeni seviye başlar. Noktalar yeniden yüklenir, karakterler başlangıç konumlarına alınır ve hayaletlerin hızı bir miktar artırılarak (belirli bir üst sınıra kadar) oyunun zorluğu kademeli olarak yükseltilir.

7.4. Yüksek Skor

Yüksek skor bir metin dosyasında saklanır. Oyun açıldığında bu dosyadan okunur, oyun sırasında güncellenir ve oyun bittiğinde ya da programdan çıkıldığında tekrar dosyaya yazılır. Böylece yüksek skor oturumlar arasında kalıcı olur.

7.5. Ses ve Arayüz

- Ses efektleri: nokta yeme, güç meyvesi, hayalet yeme, ölüm ve kovalama sireni
- Üst bilgi çubuğu: anlık puan, yüksek skor ve seviye numarası
- HAZIR! ekranı: oyun başında ve her yeniden doğuşta kısa bir bekleme süresi

8. FONKSİYONLAR

Aşağıdaki tabloda projede kullanılan başlıca fonksiyonlar ve görevleri listelenmektedir.

Fonksiyon	Görevi
applyTunnel	Labirentin iki yanındaki tünel geçişini yönetir; bir kenardan çıkan koordinatı karşı kenara sarar.
isTunnelJump	Bir hareketin normal adım mı yoksa tünelden ışınlanma mı olduğunu belirler.
distanceSq	İki kare arası uzaklığın karesini hesaplar; hızlı mesafe kıyası için.
getBlinkyNextStep	BFS ile Pac-Man'e en kısa yoldaki ilk adımı döndürür.
getPinkyNextStep	Pac-Man'in 4 kare önünü hedefleyerek BFS ile yol bulur .
getInkyNextStep	Blinky'nin konumuna göre vektör hesaplayarak hedef belirler .
getClydeNextStep	Mesafeye göre kovalama veya köşeye çekilme kararı verir.
getFleeNextStep	Kaçış modunda Pac-Man'den en uzak komşu kareyi seçer.
countDots	Haritada kalan nokta ve güç meyvesi sayısını sayar.
resetActors	Oyuncu ve hayaletleri başlangıç konumlarına döndürür.
advanceLevel	Tüm noktalar bitince yeni seviyeyi başlatır; haritayı yeniler, hızı artırır.
triggerDeath	Ölüm animasyonunu başlatır ve ölüm sesini çalar.
eatOrDie	Hayalet-Pac-Man çarpışmasını çözer; kaçış modunda yer, normalde öldürür.
saveHighScore	Yüksek skoru dosyaya kaydeder.
drawHUD	Puan, yüksek skor ve seviye bilgisini ekrana çizer.

Tablo 5. Temel fonksiyonlar ve görevleri

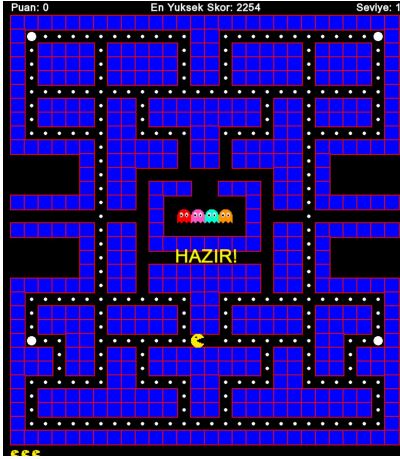
9. KARŞILAŞILAN SORUNLAR

- Yeneni hayaletin tekrar tekrar yenmesi: Kaçış modunda hayalet üzerine gelindiğinde hayalet yerinde kaldığı için aynı hayalet birden çok kez yeniyordu. Çözüm olarak çarpışma denetimi tek bir merkezde toplandı ve yeneni hayalet anında başlangıç (ev) konumuna gönderildi.

- Ses efektinin sürekli baştan başlaması: Nokta yeme sesi her yemede yeniden tetiklendiğinden ses hep baştan kesiliyordu. Çözüm olarak ses yalnızca çalmıyorken yeniden başlatılacak biçimde düzenlendi.
- Pencere boyutu sorunu: Sabit 1920×1080 pencere, daha küçük ekranlarda işletim sistemi tarafından küçültülünce labirent yanlış konumlanıyordu. Çözüm olarak pencere boyutu, labirentin boyutuna göre çözünürlükten bağımsız hesaplanacak biçimde değiştirildi.
- Arayüz yazılarının labirentle çakışması: Üst bilgi çubuğu labirentin üstüne biniyordu. Çözüm olarak labirentin üstüne ayrı bir boşluk eklendi.

10. OYUN İÇİ GÖRSELLER

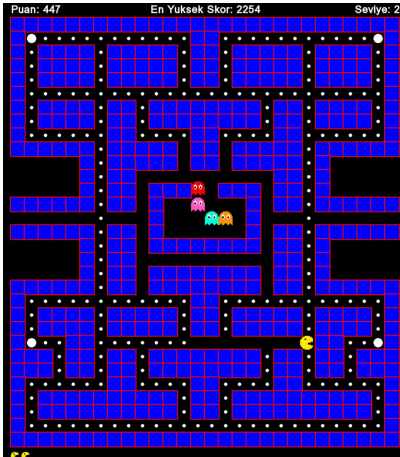
Bu bölümde oyunun farklı durumlarına ait ekran görüntüleri yer almaktadır.



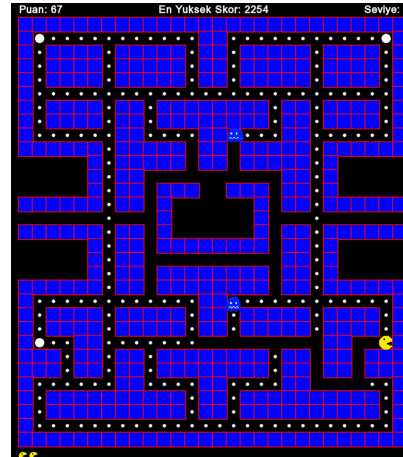
Oyun başlamadan önce (HAZIR! ekranı)



Can hakkı bittiğinde gelen Oyun Bitti ekranı



Yeni seviyeye (Seviye 2) geçiş



Güç meyvesi yendiğinde kaçan hayaletler